FACETRACK AI

Smart Attendance Management & Real-Time Target Biometrics System

Project Domain: Artificial Intelligence / Computer Vision	**Architecture:** Flask MVC + WebRTC Client HUD
Primary Backend: Python 3.11+ / OpenCV / Dlib	**Storage:** SQLite Relational Database
Date of Report: October 10, 2026	**Status:** Production Ready / Tested

1. Executive Summary & Abstract

FaceTrack AI is a high-performance, commercial-grade Computer Vision application engineered to automate two critical operational challenges: **Real-Time Target Face Tracking** and **Contactless Student Attendance Management**. Built using Python, Flask, OpenCV, Dlib, and SQLite, FaceTrack AI captures video frames directly from client browser WebRTC video streams without third-party plugins. The system decodes incoming frames, computes 128-dimensional facial metric embeddings via a Deep Residual Network, matches identities using Euclidean distance against enrolled profiles, and executes simultaneous **liveness and facial feature analysis** (Eye Blink detection, Smile detection, Emotion classification, and Surgical Face Mask verification).

To ensure continuous reliability across diverse computing environments, the system features a **Dual-Engine Fallback Pipeline**: it utilizes Dlib's 68-point facial landmark model when available, and automatically cascades to OpenCV's Haar Cascade with spatial color-moment feature descriptors on minimal CPU configurations, guaranteeing zero application crashes.

2. Problem Statement & Motivation

Organizations and educational institutions face severe limitations with legacy attendance and surveillance methods:

- **Manual Registers:** Incur massive administrative labor, high error rates, paper waste, and proxy manipulation.
- **Fingerprint Biometrics:** Require physical contact (unhygienic), suffer sensor degradation, and cause long queue bottlenecks.
- **RFID Card Swiping:** Highly susceptible to 'buddy-punching' (students scanning badges for absent peers) and lost cards.
- **Heavy Surveillance Solutions:** Require expensive proprietary GPU servers, native desktop agents, and dedicated IP cameras.

FaceTrack AI solves these issues by delivering zero-touch biometric verification through ordinary webcams, browser-level canvas HUD rendering, anti-spoofing liveness checks, and instant administrative report exports.

3. System Architecture & Technical Flow

FaceTrack AI implements a modular **Model-View-Controller (MVC)** architecture divided into four decoupled tiers:

Tier	Component	Key Responsibilities & Technologies
Presentation	HTML5 / Canvas / WebRTC	Captures live video feed (2-4 FPS), renders interactive HUD bounding boxes, coordinates voice speech alerts.

Application	Flask Blueprints (`routes/`)	Exposes REST JSON endpoints (`/api/recognize_frame`, `/api/upload_target`), handles session routing, and renders templates.
AI Engine	`services/face_service.py`	Frame decoding, face detection (HOG/Haar), 68-point landmarks, 128D embeddings, Euclidean vector matching, and liveness scoring.
Data Tier	SQLite (`database.db`)	Relational schema with foreign keys, storing target profiles, serialized JSON embeddings, detection audit logs, and settings.

Data Flow Sequence: *Client WebRTC Video → Canvas Snapshot (Base64 JPEG) → HTTP POST to /api/recognize_frame → NumPy Buffer Decode → Face Bounding Box → 128D Embedding Extraction → Vector Distance vs DB Encodings → Liveness/Emotion Math → JSON Response with Coordinates → Client Canvas Draws Green/Red Target Tag & Plays Audio.*

4. Computer Vision & Mathematical Formulations

The biometrics pipeline operates on mathematically sound computer vision algorithms:

4.1 128-Dimensional Deep Metric Embeddings & Vector Distance

A deep residual network (ResNet) projects the aligned face into a 128-dimensional Euclidean space where distances correspond to facial similarities. The similarity between a probe face embedding vector *e_probe* and a registered target embedding *e_known* is measured using Euclidean Distance (L2 norm):

Distance Formula: $d = \sqrt{(\sum(e_known[i] - e_probe[i])^2)} \quad for\ i \in [1, 128]$

The system uses a configurable tolerance threshold θ **= 0.50**. Matches with $d \leq \theta$ are accepted. The confidence percentage formula is derived as:

Confidence Formula: $Confidence\ (\%) = 100.0 - (d\ /\ (2 \times \theta)) \times 50.0 \quad (clamped\ between\ 0.0\%\ and\ 99.9\%)$

4.2 Liveness Verification & Anti-Spoofing Math

To prevent photo replay attacks and provide rich behavioral analytics, the system extracts 68 facial landmarks:

- **Eye Aspect Ratio (EAR) for Blink Detection:** Evaluates vertical eyelid distance over horizontal eye width:
  EAR = (||p2 - p6|| + ||p3 - p5||) / (2.0 × ||p1 - p4||)
  If EAR < 0.22, an active eye blink is logged, proving genuine biological presence.
- **Mouth Aspect Ratio (MAR) & Lip Corner Elevation:**
  Calculates corner elevation Lip_Center_Y - (Left_Corner_Y + Right_Corner_Y)/2. A positive lift (> 0.4px) or mouth width ratio (> 0.38) triggers active smile detection.
- **Emotion Classification:** Evaluates inner eyebrow furrow ratio and eyebrow-to-eye vertical gaps to classify five distinct psychological states: *Happy, Surprised, Focused, Sad,* and *Neutral*.
- **Surgical Face Mask Detection:** Crops the lower 40% of the face, converts to HSV color space, checks for surgical blue hues ($H \in [80, 130]$), and tests grayscale standard deviation ($\sigma < 18.0$) to flag face coverings.

5. Database Schema & Data Models

The relational database (database.db) is managed through SQLite with foreign keys enforced and optimized indexes:

Table Name	Primary Key	Key Fields & Constraints	Description

targets	id (Auto)	target_id (UNIQUE, TGT-XXXX), name, notes, photo_path, status, created_at	Stores registered tracking identities and picture references.
target_encodings	id (Auto)	target_id (FK → targets), encoding_json (TEXT, 128 floats)	Stores serialized 128-float face embeddings per target.
detection_logs	id (Auto)	target_id, target_name, timestamp, date, time, confidence, emotion, smile_detected, blink_detected, mask_detected, location_tag	Audit trail of all face recognition events. Indexed on date and target_id.
settings	key (TEXT)	value (TEXT)	System parameters: tolerance, audio_alert, cooldown_seconds.
students	id (Auto)	student_id (UNIQUE), name, roll_number, email, phone, department, semester	Academic student registry for class attendance module.
attendance	id (Auto)	student_id, date, time, status (Present/Late), method, confidence	Daily class attendance records and check-in times.

6. RESTful API Specification

FaceTrack AI exposes lightweight, high-speed JSON endpoints for frontend integration and third-party systems:

Endpoint	Method	Payload Description	Response
/api/recognize_frame	POST	JSON: {'image': '...'}	JSON with matched targets, coordinates, confidence %, and liveness flags.
/api/upload_target	POST	Multipart form: name, photo_file OR base64 snapshot.	Extracts encodings, generates ID (e.g. TGT-8B3F11), returns 200 OK.
/api/stats	GET	None	Real-time counters: total targets, spotted today, last detection timestamp.
/api/clear_logs	POST	None	Truncates detection history records.

7. UI/UX & Engineering Highlights

- **Custom Glassmorphic Interface:** Styled with tailored HSL tokens, smooth dark theme backdrop, Outfit typography, and frosted glass cards (backdrop-filter: blur(16px)).
- **Dynamic Canvas HUD:** Accurately overlays bounding boxes, identity tags, and status rings over the live WebRTC stream, automatically scaling between native camera resolutions (e.g., 1280x720) and browser display viewports.
- **Interactive Voice Alerts:** Leverages client-side Web Speech Synthesis (speechSynthesis.speak) to vocalize: *'Target Identified: [Name]'*, giving security operators eyes-free situational awareness.
- **Smart Anti-Flooding Cooldown:** Employs an in-memory timestamp hash combined with a SQL cooldown check (60s default) to prevent database record flooding when a person remains in camera view.
- **Automated Exporting Engine:** Compiles zebra-striped PDF attendance sheets via ReportLab and formatted Excel workbooks via OpenPyXL/Pandas with a single click.
- **Mobile Camera & Cross-Origin Support:** Includes pre-configured SSL certificates (cert.pem, key.pem) allowing smartphones on the same Wi-Fi network to grant WebRTC camera access over HTTPS.

8. System Requirements & Tech Stack

Category	Minimum Requirement	Recommended Specification
Processor (CPU)	Dual-Core 2.0 GHz Intel/AMD or Apple Silicon	Quad-Core 2.5 GHz+ (Intel i5/i7/Apple M-series)
RAM	4 GB DDR4	8 GB+ DDR4 / Unified Memory
Camera Sensor	720p HD Webcam (30 FPS)	1080p Full HD Wide-Angle Camera
Operating System	Linux (Ubuntu 20.04+), macOS 12+, Windows 10/11	macOS Sonoma / Ubuntu 22.04 LTS
Python Environment	Python 3.10+	Python 3.11 / 3.12 with OpenCV & Dlib compiled

9. Future Enhancements & Conclusion

Future Scope:

1. **Multi-Camera RTSP Streams:** Ingest continuous video feeds from multiple distributed IP security cameras.
2. **GPU TensorRT Acceleration:** Deploy NVIDIA TensorRT for real-time batch processing of 30+ faces per frame at 60 FPS.
3. **Active Liveness Challenges:** Introduce randomized prompts (e.g., 'Turn head 30° right', 'Blink twice') for high-security areas.

Conclusion:

FaceTrack AI demonstrates a robust, production-ready implementation of modern computer vision. By combining deep metric face embeddings, real-time liveness analysis, anti-flooding rate limiting, and an intuitive browser-based Glassmorphism UI, the project delivers a secure, touchless, and versatile biometric surveillance and attendance platform.